

The Cognitive MMU

The CCOS Project¹

2026-06-26

Abstract

An LLM coding agent’s working memory is managed opaquely: a retrieval stack (RAG, GraphRAG, agent frameworks) pages a repository into a bounded window and mutates that window probabilistically as a session unfolds, so when an agent drifts after dozens of steps it is impossible to say *why* or *when* its representation of the project corrupted. CCOS treats agent memory the way an operating system treats RAM: source is parsed into an *event-sourced* causal graph, nodes are scored and paged under a token budget, and every state transition is recorded in a hash-chained log that replays bit-for-bit. We give a precise, deterministic definition of a *causal region* (a connected component of the cross-file causal-link graph) and of causal distance (a weighted shortest path), and prove that regions and the paged window are a pure function of the graph, reconstructing exactly on replay — the property that makes an agent’s memory *auditable* and *time-travel debuggable*. We then ask whether this causal structure makes CCOS a *better retriever* than a baseline, and report — honestly — that it does not. Measured on 70 real bug-fix commits across three mature crates, causal selection **ties a plain lexical TF-IDF retriever** and loses to it at a tight budget; on real bugs a fix’s files share vocabulary, so lexical similarity finds them too, and a proof-of-concept that seeds CCOS from a crash trace is likewise beaten by RAG-over-the-error-message (Rust error messages name the cause). An LLM-free simulation *does* separate the two when a cause is constructed to be lexically dissimilar from its symptom, and a first on-device run with a 7B model reproduces it — but that regime does not occur on real code. The contribution is therefore *not* better retrieval. It is the infrastructure a retrieval stack structurally lacks: a *deterministic, replayable, auditable* agent memory in which one can rewind the exact context state at any step and replay it under different parameters (*time-travel debugging*). To our knowledge CCOS is the first system to treat an agent’s *working memory itself* as a transactional subsystem — deterministic, hash-chained, replayable bit-for-bit, and *post-mortem debuggable*: a flight recorder for attention, with a breakpoint on the exact step at which the right information was evicted from the window. We release the engine, the honest validation harness, and this paper, labelling throughout what is *proven* (formal structure, determinism, replay), *measured* (retrieval coverage — where CCOS does *not* beat lexical RAG on real bugs), and the one axis where CCOS *does* win: **efficiency**. Run end-to-end on real single-file fixes with a 30B model and a compiler-in-the-loop retry, CCOS and RAG resolve *equally* (2/10), but CCOS does so on 776 context tokens against the lexical retriever’s 5366 — a $6.9\times$ reduction (and $4.1\text{--}9.1\times$ across 51 fixes from three crates, model-free), because the causal region stops at the working set rather than padding a top- k to budget. CCOS is thus not a better retriever or solver, but a more *frugal, auditable* one — it self-calibrates, with no k to tune. (On its own source tree, region selection covers a task’s k -hop neighbourhood with recall 0.97 vs 0.35 flat at $\approx 48\%$ fewer tokens; regions are 95.5% internally connected.)

Introduction

An LLM coding agent operating over a repository must repeatedly decide *what to put in the context window*. The dominant pattern is retrieval: embed code chunks, rank them against the task, and concatenate the top- k until a token budget is exhausted. This treats context as a one-dimensional ranked list. Two failure modes follow. First, *dilution*: as tasks lengthen, the window accumulates globally salient but task-irrelevant code, and models attend poorly to the middle of long contexts. Second, *incoherence*: the top- k chunks need not form a connected unit of reasoning — a function may be paged in without its callers, its error sites or the data it depends on.

Operating systems faced the analogous problem decades ago and answered with the *working set*: the set of recently referenced pages that must stay resident. CCOS adopts the analogy for LLM context: source is parsed into a causal graph, nodes are scored, and a bounded “context window” is paged in and out like RAM $\leftrightarrow$ VRAM. Unlike a retrieval stack, every transition is recorded in a hash-chained event log, so the memory is not a black box but an *auditable, replayable* artefact. Our contributions are:

1. A **formal, deterministic model of context regions** (§4, §5): causal distance as a weighted shortest path, region membership as a connected component of the cross-file causal-link graph, and a determinism theorem — regions and the paged window are a pure function of the graph, so a session reconstructs bit-for-bit from its hash-chained event log (tamper-evident replay).
2. **Event-sourced agent sessions with time-travel debugging** (§6): every cognitive operation (ingest, failure signal, recall) is logged; the exact context state at any step is reconstructible; and a recall can be *replayed under different parameters* to ask whether the agent would have decided better — the capability a probabilistic retrieval stack structurally lacks. To our knowledge this is the first treatment of context assembly *itself* as a replayable, post-mortem-debuggable subsystem — a *flight recorder for an agent’s attention* — with an *eviction watchpoint* that names the exact step and operation at which the true cause was squeezed out of the budgeted window.
3. An **honest validation harness and a negative result** (§10): on 70 real bug-fix commits, causal selection *does not* beat a lexical TF-IDF retriever at placing a fix’s files in the window (it ties, and loses at a tight budget), and a crash-trace pivot is beaten by RAG-over-the-error-message. We report this plainly — it relocates CCOS’s value from *retrieval* to *auditability*.
4. An **LLM-free locality measurement** (§8) with real, reproducible numbers, and a falsifiable *sufficient-condition* protocol (Phase 4) we specify but leave open.

Related Work

Retrieval-augmented generation.

RAG augments a parametric model with a non-parametric store and retrieves passages per query. Self-RAG adds reflection tokens that gate retrieval and critique generations. These operate on *independent* chunks; coherence between retrieved items is not modelled.

Graph- and structure-aware retrieval.

GraphRAG builds an entity knowledge graph and answers global queries by summarising community structure . For code, property graphs unify AST, control flow and data flow . CCOS regions are in this lineage but target *paging*: which connected sub-structure to make resident for a task, under a token budget, with deterministic eviction.

Agent memory.

MemGPT casts the LLM as an OS that pages between an in-context window and external storage ; Generative Agents retrieve memories scored by recency, importance and relevance — the same factors CCOS aggregates into a region temperature. LangGraph structures agents as stateful graphs of steps; it orchestrates *control* flow, whereas CCOS structures *memory*. The two are complementary.

Context-window management.

KV-cache eviction keeps “heavy hitter” or sink tokens resident , and PagedAttention applies OS paging to the KV cache . These act at the token level inside one forward pass; CCOS acts at the *semantic* level across an agent session.

The Causal Context Substrate

CCOS parses source into a directed *causal memory graph* $G = (V, E)$. A node $v \in V$ is a file, module, symbol or external dependency, with scalar fields $\text{imp}(v)$ (base importance), $\text{fail}(v) \in [0, 1]$ (failure relevance) and $\text{rec}(v) \in [0, 1]$ (recency). An edge $e = (u \rightarrow w) \in E$ carries a weight $w(e) \in (0, 1]$ and a type (containment, dependency, reference, causation). The kernel assigns each node a causal score $\text{score}(v) = \text{clamp}(0.15 \text{ imp}(v) + 0.50 \text{ fail}(v) + 0.30 \text{ rec}(v) + 0.05 \ln(1 + \text{acc}(v)), 0, 1)$, where $\text{acc}(v)$ is the access count. Faults propagate along edges, fail decays with a logical clock, and every state transition is appended to a hash-chained event log enabling deterministic replay . Node identifiers are namespaced (`file:p`, `mod:p:n`, `use:p:path`, `sym:p:n`, `dep:root`); the owning file of a node is recoverable from its identifier. CCOS v0.2 pages in the top-score nodes: a flat, 1-D policy. We now make selection spatial.

Context Regions

Causal distance

Definition 1 (Causal distance). Let $\hat{G}$ be the undirected multigraph on V induced by E , and assign each edge the cost $c(e) = -\ln w(e) \geq 0$, so that a stronger causal link is a shorter step. The causal distance $d_c(u, v)$ is the minimum total cost over all u - v paths in $\hat{G}$, and $+\infty$ if none exists. The unweighted hop distance $\text{hops}(u, v)$ is defined analogously with unit costs.

$c(e) \geq 0$ since $w(e) \leq 1$, so d_c is a genuine shortest-path metric on each connected component (non-negative, symmetric, triangle inequality). The k -hop causal neighbourhood of a target t is

$$\mathcal{N}_k(t) = \{v \in V: \text{hops}(t, v) \leq k\}.$$

$\mathcal{N}_k(t)$ is the ground truth “what is causally relevant to a task at t ” used in the evaluation (§8).

Region membership

External dependency hubs (e.g. `dep:std`) connect almost everything and must not collapse the graph into one component. We therefore separate them.

Definition 2 (Cross-file causal link). *For a non-external node v let $\phi(v)$ be its owning file. Two files f, g are directly linked, written $f \approx g$, iff there exists an edge $(u \rightarrow w) \in E$ with $\phi(u) = f$, $\phi(w) = g$, $f \neq g$, and neither u nor w is an external dependency node.*

Definition 3 (Region). *Let $\approx^*$ be the reflexive-transitive closure of $\approx$ on the set of file keys. A region is the set of all nodes whose files lie in one equivalence class of $\approx^*$; all external dependency nodes form one additional region. Two nodes belong to the same region iff their files are in the same connected component of the cross-file causal-link graph.*

Proposition 1 (Regions partition the nodes). *The regions of Definition 3 form a partition of V : every node lies in exactly one region.*

Proof. $\approx^*$ is an equivalence relation on file keys, so its classes partition the file keys; the external bucket is a distinct class by construction. Mapping each non-external node to its file’s class and each external node to the external class is a total function into disjoint blocks, hence a partition. $\square$

By default (only containment and import edges) each source file is its own region. A genuine cross-file dependency or a propagated failure merges files into a single multi-file region — the “zone of knowledge” an agent should wake together.

Region scalars

For a region R with member set M :

$$\text{heat}(v) = 0.5 \text{score}(v) + 0.3 \text{fail}(v) + 0.2 \text{rec}(v),$$

$$\text{temp}(R) = \text{clamp}\left(\frac{1}{|M|} \sum_{v \in M} \text{heat}(v), 0, 1\right),$$

$$\text{dens}(R) = \frac{|\{e \in E: \text{both endpoints} \in M\}|}{|M|}.$$

Temperature is how “awake” a region is; density is its internal causal cohesion (internal edges per member). Activation warms a region ($\text{temp} + 0.25$, capped) and records a logical tick; a cooldown step multiplies temperatures by a decay factor and evicts regions below a floor.

Dynamic admission policy

The static threshold 0.6 becomes a function of token pressure $u \in [0, 1]$ (the used fraction of the budget) and task complexity $\kappa \in [0, 1]$:

$$\begin{aligned}\theta(u, \kappa) &= \text{clamp}(0.6 + 0.3 u - 0.2 \kappa, 0.05, 0.95), \\ a(R) &= 0.55 \text{temp}(R) + 0.30 \frac{\text{dens}(R)}{1 + \text{dens}(R)} + 0.15 \kappa, \\ \text{admit}(R) &\Leftrightarrow a(R) \geq \theta(u, \kappa).\end{aligned}$$

A hot, cohesive region can be admitted even when the static 0.6 would reject it; a nearly-full window raises θ so only the hottest regions enter.

Determinism and Replay

Theorem 1 (Regional determinism). *The region partition, every region scalar in [eq:temp]–[eq:dens], and the activation history are pure, order-independent functions of the graph G and the sequence of (logically timestamped) activation events. Consequently, given the event log of a session, the engine state reconstructs identically: if G' is the graph rebuilt from the log and L its region events, then $\text{replay_from}(G', L)$ equals the live engine that produced L .*

Proof sketch. Clustering enumerates nodes and edges in sorted order and computes connected components by a sorted breadth-first search, so its output is independent of hash iteration order. Equations [eq:score], [eq:temp] and [eq:dens] are deterministic arithmetic over node fields and a count of qualifying edges. Activation reads a logical clock (a counter), never wall-clock time, and the emitted RegionActivated event records the exact tick. Reconstruction re-clusters G' (identical base state, since $G' = G$ structurally) and applies the recorded activations in log order; each step is the same pure function of the same inputs, so the resulting state is identical. The integration test `replay_reconstructs_identical_engine` checks `engine = replay_from( $G', L$ )` by structural equality.  □ □

This extends CCOS’s existing guarantees: the primary event log is hash-chained and tamper-evident, so the region history is auditable, and a 10 000-cycle test exhibits no drift in region count or temperatures.

An auditable input boundary.

The same deterministic, replayable substrate hardens the text an agent ingests. Hidden-character injection vectors — bidirectional overrides (the *Trojan Source* attack, CVE-2021-42574), zero-width formatting, and the Unicode *Tags* block used for invisible ASCII smuggling — are de-obfuscated at ingest into explicit, visible literals ([U+202E RL0]) rather than silently stripped, and the findings are recorded against the same hash-chained log, so a replay reproduces the cleaned state. A downstream linear log-space classifier — the closed form of multinomial Naive Bayes, $\text{logit} = \mathbf{b} + \mathbf{W} \cdot \mathbf{X}$ over a hashing-trick feature vector, its weights locked in a checksum-verified blob — adds a *deterministic, forensically decomposable* injection signal (a held-out red-team measures $F_1 = 0.90$; precision 0.87, recall 0.93). We are deliberate that this is a *signal*, not a *solution*: a bag-of-features model is evaded by paraphrase, and no character-level pass addresses semantic injection — the real mitigation is privilege separation in the host. The contribution is not a novel detector but that de-obfuscation and scoring inherit CCOS’s defining property: every security decision is reproducible and auditable down to the exact feature that moved it.

Event-sourced sessions and time-travel debugging

Determinism is not only a correctness property; it is the basis of the capability that, our evaluation will argue, distinguishes CCOS from a retrieval stack. An `AgentSession` records every cognitive operation an agent performs on its memory — `Ingest`, `SignalFailure`, `Recall` — as an ordered timeline. Because each operation is a deterministic function of the prior state, the memory after the first n operations is reconstructed exactly by replaying the mutating ones (`replay_to(n)`): one can *rewind the agent's mind* to any step.

The operation that matters for debugging is the counterfactual. `recall_what_if( $n$ ,  $q$ ,  $b$ )` replays the memory to step n and re-runs a recall with a different query q or token budget b , returning the window the agent *would* have seen. When an agent emits a bad patch at step 15, an operator can replay its exact context at step 14, widen the budget or change the anchor, and observe whether the decision improves — a closed-loop, reproducible debugger for an agent's working memory.

A breakpoint on attention.

Replay also localises *drift*. The missing watchpoint scans the timeline for the exact step at which a given node — typically the true cause of a failure — was squeezed out of the budgeted window by competing pressure, naming the triggering operation and the token gap; a complementary energy view surfaces the causal-heat migration through the graph that a file-level diff misses. It is, in effect, a *breakpoint on an agent's attention*: the precise moment the right information left the window. An opaque retrieval stack can show only the final, corrupted context; CCOS can point to the step — and the operation — at which it went wrong. A RAG or framework stack mutates its store probabilistically across a session and keeps no canonical, replayable transition log, so the same question (*why, and when, did the representation corrupt?*) has no answer there. We do not claim this improves task success; we claim it is a property the alternatives do not have, and that it is the honest locus of CCOS's contribution (§10).

The log as training data.

Because the timeline is deterministic and replayable, it is also a *counterfactual evaluation set* for retrieval itself. We read a reward straight off it: for each recorded recall, was the node the agent engaged *next* — a failure signal or page-fault, taken as a post-hoc relevance signal — present in the window that recall *would* have produced under candidate scoring weights? Maximising this hit rate by deterministic coordinate ascent (each candidate scored by a full replay) yields scoring weights tuned to the session's own history; adopting them is recorded as an operation, so the learned policy is itself auditable and reproduced on replay. The reward is an honest proxy and the optimiser is greedy, so we claim only that this is retrieval that *trains on a substrate the alternatives lack*: a probabilistic RAG store keeps no canonical, replayable log to learn from. This closes the loop the title implies — the working memory is not only inspectable but *self-improving* from its own auditable record.

Implementation

The engine is ≈ 600 lines of dependency-light Rust layered above the graph, with no change to the parser, guard, incremental builder or event-sourcing core. The public surface is: `ContextRegionEngine` (`cluster_nodes`, `initialize_regions`, `activate_region`, `tick_cooldown`, `replay_from`), the data types `ContextRegion` / `ContextPoint` / `ContextWindow`, the `ContextPolicy`, and five new event variants (`RegionCreated`, `RegionActivated`, `RegionMerged`, `RegionEvicted`, `ContextWindowGenerated`). A `ccos` regions CLI exposes clustering, activation and the locality report. The whole crate builds warning-clean under `clippy -D warnings` and passes 431 tests.

The cognitive MMU, made real.

Paging lives in the code, not only the prose: eviction is *non-destructive*. When the resident node set exceeds its cap, the lowest-scored nodes are *demoted*—with their incident edges—into a COLD tier rather than dropped, and any node faults back on demand (`page_in`). A failure signal, or a recall *around* a demoted node, resurrects it and its cold neighbours transparently; the page-in is a *replayed* side effect, so `replay = live` still holds. The COLD tier’s unbounded *content* optionally spills to a content-addressed on-disk store (SHA-256 keys, deduplicated, and *hash-verified* on read—a tampered or missing blob is a cold-miss, never a silent empty restore), bounding the resident-plus-cold *content* footprint in RAM while the backing store on disk is unbounded. At the deepest tier, once the backing store *itself* must stay frugal, an optional budget *compacts* the coldest content—code skeletonised, prose summarised—to a causal summary, discarding the original. This is genuine, *auditable forgetting*: it is lossy, but observable (`cold_compacted`) and never a silent drop. This is the sense in which working memory is “unbounded”: a *direction*, frugality $\times$ available RAM, rather than a literal claim—and at the floor frugality is the master, exactly as the demand-paging analogy demands. Honestly scoped: spill moves only *content* to disk (per-cold-node metadata still grows in RAM—an on-disk index is future work), blobs are stored verbatim (dedup, no compression codec yet), and compaction bounds the cold *content* footprint, not the entry *count*. Both spill and compaction are opt-in; the default path keeps them off and serialization byte-identical, so the determinism guarantees of §5 are untouched.

Hybrid entry fusion.

A free-text recall resolves its entry node by *reciprocal-rank fusion* of three independent rankings—lexical token overlap, semantic INT4-TF-IDF cosine, and a causal *active-failure* focus—before the usual region expansion. RRF compares ranks, not raw scores, so the three incomparable signals fuse without calibration. The causal vote is deliberately *sparse*: it ranks only nodes under failure pressure, so it abstains on a quiet graph (no spurious id-ordered bias when scores are flat) and speaks for the active problem region once a failure is signalled—the CCOS-native attention signal rather than a generic importance prior. Deterministic, and an honest improvement to *entry selection* only: the downstream region expansion and the evaluation of §8 are unchanged. The semantic signal itself is, by default, deterministic INT4 TF-IDF; an opt-in build distils it into a learned *latent-semantic* projection (LSA / truncated SVD of the corpus’s document–term matrix, via a fixed Jacobi sweep), which captures synonymy raw TF-IDF cannot while remaining zero-dependency and deterministic — the “distil, don’t couple” rule that lets a learned component respect the replay invariant.

Evaluation: what we can measure today

We separate *measured* results (this section, fully reproducible and LLM-free) from *hypothesised* agent-level gains (§10).

Setup.

We run on CCOS’s own source tree ($|V| = 705$ nodes, $|E| = 822$ edges, yielding 26 regions). For a target node t we take $\mathcal{N}_k(t)$ with $k = 2$ as ground truth and compare two selection strategies at the region’s size budget: **flat** — the top- $|R|$ nodes by score (CCOS v0.2 / classical ranked retrieval) — and **region** — the members of t ’s region. We report causal precision $|\mathcal{S} \cap \mathcal{N}_k|/|\mathcal{S}|$, recall $|\mathcal{S} \cap \mathcal{N}_k|/|\mathcal{N}_k|$, and the tokens each strategy needs to cover $\mathcal{N}_k$. All numbers are emitted by `scripts/region_benchmark.sh` and are deterministic.

Locality on `src/` ($k = 2, 12$ targets). Region selection covers 97% of a task’s causal neighbourhood versus 35% for flat, at $\approx 48\%$ fewer tokens for equal coverage.

Strategy	causal precision	causal recall
flat (v0.2 / ranked)	0.021	0.347
region (v0.3)	0.057	0.972

Cohesion and cost.

Regions reach an **average causal density of 0.955** (Eq. [eq:dens], normalised) — they are almost entirely internally connected, i.e. genuine causal clusters rather than arbitrary file groups. Building the region map for 705 nodes takes ≈ 20 ms (54.5 builds/s); a single activation is ≈ 20 ms.

Honest reading.

Absolute precision is low (0.06) because the v0.2 parser emits only containment and import edges, so $\mathcal{N}_k(t)$ is tiny (often 2–3 nodes) while a region spans a whole file. The robust, real wins are *recall* (0.97 vs 0.35) and *token efficiency* ($\approx 48\%$): a region reliably contains a task’s causal neighbourhood and pays fewer tokens to do so. Richer semantic edges (call graph / data flow, roadmap item P1.3) would sharpen $\mathcal{N}_k$ and is the main lever to raise precision; we flag this rather than hide it.

Hypothesis simulation under a stated oracle (LLM-free)

The core thesis — *does regional causal memory help an agent on long, multi-file tasks?* — ultimately needs LLM rollouts (§10). But its *necessary condition* is testable now, without an LLM: **an agent cannot solve a task whose required causal context is absent from its window**. We measure, under an explicit oracle, whether each selection strategy *places the required causal context in the window*.

Setup.

We generate modular synthetic repositories: S independent subsystems, each a set of files linked by one cross-file causal chain plus high-score *decoy* symbols; there are no edges between subsystems, so each subsystem is one bounded causal region. The causal structure is *decoupled* from the lexical structure — a chain neighbour shares no identifier token with the target, only an edge — modelling a dependency that is causally essential yet lexically dissimilar. A task of *diameter* d requires the $\pm d$ window along its chain (up to $2d + 1$ files); the budget holds $\approx$ one subsystem, far less than the repository. The **oracle** is $R(t) \subseteq S$.

Crucially, retrieval pipelines locate code from a text *query*, whereas an OS-style memory *anchors* on the workspace signal (the active file, a failing test). We model both: each task carries a query (a token bag) and an anchor (the active file node), and we run two scenarios — **clean** (the query points at the target) and **noisy** (a trap decoy in an unrelated subsystem out-scores the target lexically). Six strategies share the budget: rag-dense (top- k lexical), rag-hybrid (lexical + causal score), graphrag-1hop (best hit + one hop), graphrag-bfs (unbounded expansion from the best hit), ccos-from-query (CCOS region of the best *lexical* hit — an ablation), and ccos-region (CCOS region of the *anchor*). Seeded and deterministic; reproduce with ccos experiment.

Hypothesis simulation (800 tasks, seed 42, budget $\approx$ one subsystem). Success = the required causal set $R(t)$ is inside the window. Under a clean query every structure-aware method ties; under a misleading query only the workspace-anchored region survives.

	success at diameter d				overall	
2-5(lr)6-7 Strategy	$d = 1$	$d = 2$	$d = 3$	$d = 4$	succ.	cov.
<i>Clean query (points at the target)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	0.19
rag-hybrid	1.00	0.00	0.00	0.00	0.23	0.65
graphrag-1hop	1.00	0.00	0.00	0.00	0.23	0.58
graphrag-bfs	1.00	1.00	1.00	1.00	1.00	1.00
ccos-from-query	1.00	1.00	1.00	1.00	1.00	1.00
ccos-region	1.00	1.00	1.00	1.00	1.00	1.00
<i>Noisy query (a decoy out-scores the target lexically)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	0.19
rag-hybrid	1.00	0.00	0.00	0.00	0.23	0.65
graphrag-1hop	0.00	0.00	0.00	0.00	0.00	0.19
graphrag-bfs	0.00	0.00	0.00	0.00	0.00	0.00
ccos-from-query	0.00	0.00	0.00	0.00	0.00	0.00
ccos-region	1.00	1.00	1.00	1.00	1.00	1.00

Findings (and their honest limits).

(1) Lexical retrieval (rag-dense) *fails* on cross-file causal tasks (0%; coverage 0.19): similarity cannot surface causally-essential but lexically-dissimilar context — the hypothesis’s premise. (rag-hybrid’s $d = 1$ wins come from the *causal score* it borrows, not lexical similarity, which is why they persist under noise.) (2) The value of structure-aware selection *grows with diameter*: one-hop expansion solves only $d = 1$, while full causal paging (graphrag-bfs, ccos-region) solves all d — the direction of H2. (3) **The clean tie.** Under a clean query, graphrag-bfs, ccos-from-query and ccos-region all reach 1.00: the lever is causal *structure*, **not CCOS per se**. (4) **The noisy separation.** Under a *misleading* query, every method that locates code lexically collapses to 0% — including the strong graphrag-bfs *and* the ccos-from-query ablation (CCOS seeded on the query). Only ccos-region, which anchors on the workspace signal rather than the query, survives at 1.00. The ablation isolates the differentiator: it is the *anchor source* (a structural workspace signal vs. a lexical query), not the region machinery. This is the realistic regime — a task description rarely names the exact distant cause, and an OS-style memory that tracks the active working set is robust where query-time retrieval is misled. (5) **Assumptions, stated.** This credits CCOS with a reliable anchor (the active file / failing test), which an OS-level memory has but a pure retrieval pipeline does not; and it assumes *modular* repositories with separable regions (density 0.955 on real code, §8) — a monolithic giant region exceeding the budget collapses the advantage (observed, reported). (6) Throughout this is a *simulation under a stated oracle*: it tests the *necessary* (retrieval) condition, not the *sufficient* (generation) one of §10.

Real-LLM evaluation: a first on-device measurement

The simulation (§9) established the necessary condition. The sufficient condition — that an LLM agent then *solves* more tasks with regional memory than with chunk retrieval — requires model rollouts. We **implement the harness** (ccos eval) and report a **first real-LLM run** against a local model.

Implemented harness.

Each task is a tiny multi-file project encoding an *arithmetic causal chain*: a base constant in one file is transformed through a chain of one-line functions across separate files, and the question “what integer does the last function return?” is answerable *only* by reading the whole chain (the distant cause included). Grading is therefore exact-match on an integer — objective and automatable, with no code execution. The same six strategies of §9 assemble the context window under a token budget (with the clean/noisy query split), the window is sent to a model (any OpenAI-, Anthropic-Messages-, or Ollama-compatible endpoint), and we record three metrics per diameter: **task success** (correct integer), **oracle coverage** (required chain $\subseteq$ window — model-independent), and **symbol-hallucination** (the answer cites a function absent from the project).

Setup.

We run `ccos eval` on-device on an NVIDIA Jetson AGX Thor against `qwen2.5:7b-instruct` served by Ollama (temperature 0), with 20 tasks, seed 7, a 600-token budget, diameters 1–4, in both the clean and noisy regimes. Table 3 reports the run.

First real-LLM run: `qwen2.5:7b-instruct` via Ollama, on-device (NVIDIA Jetson AGX Thor), 20 tasks, seed 7, budget 600 tokens. “cov.” is the model-independent oracle coverage; “tok.” the mean input tokens. Success is an exact-match integer answer.

success at diameter d						
2-5 Strategy	$d = 1$	$d = 2$	$d = 3$	$d = 4$	cov.	tok.
<i>Clean query (names the target function)</i>						
rag-dense	0.12	0.00	0.00	0.00	0.00	519
rag-hybrid	0.00	0.00	0.00	0.00	0.00	521
graphrag-1hop	1.00	0.00	0.00	0.00	0.40	270
graphrag-bfs	1.00	0.00	0.17	0.00	1.00	402
ccos-from-query	1.00	0.00	0.17	0.00	1.00	402
ccos-region	1.00	0.00	0.17	0.00	1.00	402
<i>Noisy query (a decoy out-scores the target lexically)</i>						
rag-dense	0.00	0.00	0.00	0.00	0.00	531
rag-hybrid	0.00	0.00	0.00	0.00	0.00	521
graphrag-1hop	0.00	0.00	0.00	0.00	0.00	165
graphrag-bfs	0.00	0.00	0.00	0.00	0.00	165
ccos-from-query	0.00	0.00	0.00	0.00	0.00	165
ccos-region	1.00	0.00	0.17	0.00	1.00	402

Findings.

(1) Coverage transfers from simulation to real text. The model-independent coverage column reproduces Table 2 on real file content: lexical RAG covers 0, the structure-aware methods cover 1.00 under a clean query, and under noise *only* `ccos-region` holds 1.00 — the necessary condition, confirmed outside the simulator. **(2) Success is bounded by the model, not the context.** Where coverage is 1.00, the 7B model still solves only $d = 1$ (1.00) and part of $d = 3$ (0.17), missing $d = 2, 4$: with the whole chain in the window, the residual failure is *arithmetic reasoning* — a model-capability ceiling, not a selection failure. Crucially no method ever succeeds *without* coverage (success $\subseteq$ coverage), which is the claim the harness is built to test. **(3) The noisy separation holds on a real LLM.** Under a misleading query every query-driven method collapses to 0% success — including `graphrag-bfs` and the `ccos-from-query` ablation — while `ccos-region`, anchored on the workspace signal, is the *only* strategy with non-zero success ($d = 1$ at 1.00). The query-driven methods even look *cheaper* under noise (165 vs 402 tokens) precisely because they confidently retrieve the wrong, smaller file; `ccos-region` spends its budget on the causally-correct chain. **(4) Honest limits.** 20 tasks (≈ 5 per diameter) is a small sample, and a 7B model floors the sufficient condition. A frontier model

(deepseek-v4-pro, in progress) or a 70B local model is expected to lift success wherever coverage is 1.00, *sharpening* — not changing — the separation, since coverage already fixes the ceiling.

Toward external benchmarks.

The arithmetic-chain suite isolates the selection question; the decisive external tests are SWE-bench issue resolution (a patch passes the hidden tests) and a controlled *multi-file bug* suite where the fault site, its cause and its blast radius lie in distinct files. Baselines share the base LLM and token budget: classical RAG (top- k cosine over chunks), GraphRAG (community summaries over a code graph), MemGPT (OS-style paged memory), a LangGraph agent with a vector store, and CCOS regions. Metrics: resolved-rate, input tokens to success, and a citation-grounding hallucination rate checked against the ground-truth graph. We have begun this on real history:

`scripts/causal_validation` mines fix commits, checks out the pre-fix tree, injects the fault, and scores $R_{\text{cov}} = |F_{\text{target}} \cap \text{WorkingSet}_K| / |F_{\text{target}}|$ — the fraction of the files a fix touched that the bounded working set recovers. On this repository’s history the first run exposed a limitation and its fix, both measured: with downstream-only failure propagation R_{cov} was flat at 0.33 (only the seed file recovered, as co-changed files are *upstream* importers reached only through dependency hubs). Resolving intra-crate imports into file $\rightarrow$ file edges and propagating failure *bidirectionally* fixes this. Across three mature crates — fd, bat and hyperfine, 70 mined fix commits — the effect is consistent: at a sufficient budget ($K \geq 50$) the two changes lift R_{cov} to 0.85–1.0 (from 0.50–0.84 downstream-only), while *diluting* to 0.19–0.28 at a tight $K = 20$ budget. **But against the obvious baseline, this is not a win.** Running a classical lexical RAG (TF-IDF cosine over file text, queried by the fault file) at the *same file budget*, R_{cov} as CCOS / RAG is 0.92/0.94, 1.00/0.98, 0.87/0.92 at $K = 50$ and ties at $K = 100$, while at $K = 20$ RAG is clearly ahead (0.20/0.56 on bat, 0.20/0.73 on hyperfine). Causal selection therefore has *no net coverage advantage* over lexical similarity here, and is worse at a tight budget: on real bugs a fix’s files are lexically similar to one another, so TF-IDF recovers them too — the §9 premise of a cause lexically dissimilar from its symptom does not reproduce on these repositories. The honest reading is that the *necessary* condition holds for the large majority but is *not* CCOS-specific; a real advantage would have to come from the regimes this setup does not test — a degraded/absent query (§9), a symptom (failing test) lexically far from its cause (needing the real test-driven seed, not the highest-degree heuristic), or the *sufficient* condition (Phase 4). Multi-crate workspaces, now linkable, also remain to be measured at scale.

Sufficient condition (Phase 4): resolution ties, efficiency does not.

We run the generation half on real single-file fixes from fd: an agent is given an equal-budget context built two ways — CCOS’s causal region vs top- k lexical RAG — asked to rewrite the buggy file, and graded by whether cargo test passes, with a *compiler-in-the-loop* retry (on failure, a *context page fault* parses the error with the trace layer, injects pressure on the faulting files, and re-prompts with a refreshed window). With a weak 7B model and one shot, both resolve 0/15; with qwen3-coder-30B and three attempts the loop lifts both to 2/10 (20%) — the page-fault feedback, not the retriever, unlocks resolution, and CCOS shows **no resolution advantage** over RAG, consistent with the retrieval result. The *efficiency* separates them: at equal resolution CCOS’s auto-bounded region averages 776 context tokens against the lexical retriever’s budget-filling 5366 — a $6.9 \times$ reduction. The same holds at scale without a model: across 51 single-file fix scenarios from fd, bat and hyperfine, CCOS assembles 700–1600 context tokens against RAG’s ≈ 6000 , a $4.1\text{--}9.1 \times$ reduction. This is the one axis on which CCOS

dominates: not *what* it retrieves (a baseline matches that) but *how little* it needs, because the causal region stops at the working set instead of padding a top- k to budget. We state the caveats: the *resolution* sample is tiny ($n = 10$, two passes), and the baseline fills the budget by construction (a carefully tuned- k RAG would also be sparser). The defensible claim is narrower but real: CCOS *self-calibrates* — it bounds itself at the causal region with no k or budget to tune — so it never wastes the window, which is precisely the point of demand paging.

Hypotheses.

H1 (efficiency): CCOS reaches equal task success at fewer input tokens than RAG/GraphRAG, because a region covers the causal neighbourhood with higher recall (Table 1). **H2** (long-horizon success): on multi-file tasks CCOS’s resolved-rate exceeds chunk RAG by a margin that grows with the causal diameter of the task. **H3** (grounding): CCOS lowers the symbol-hallucination rate, because the admitted window is a connected sub-graph of *real* nodes.

Threats to validity.

Region quality is bounded by edge quality (§8); results would conflate the region *policy* with the underlying *graph*. Ablations must therefore fix the graph and vary only the selection strategy, and report per-task causal diameter so that gains are attributed to coherence, not to retrieving more text. The `ccos - from-query` ablation in Table 3 is exactly this control — same graph and budget, only the anchor swapped — and it collapses with the lexical baselines under noise. A positive result on H1–H3 would constitute the research contribution; a null result would still validate the deterministic, auditable infrastructure.

Limitations

We are deliberately explicit about what is *not* shown. **(1) No coverage advantage over RAG on real bugs.** The headline real-data metric (R_{cov} , §10) ties a plain lexical TF-IDF retriever at a sufficient budget and loses to it at a tight one; on real fix commits the files a fix touches are lexically similar, so the simulation’s premise of a lexically-dissimilar cause does not hold. The strong absolute numbers are a *necessary* condition that a baseline also satisfies, not a CCOS win. (A complementary measurement on the engine’s own source isolates a different, structurally-defined relation: a lexical TF-IDF retriever recovers only 49% (recall@10) of the AST-resolved *import* dependencies—pairs that routinely share no vocabulary, dependency cosine 0.53 vs 0.43 random—so RAG’s blind spot is real, but it lies off the *fix-cohesion* axis the headline metric measures, where a fix’s files do share vocabulary. The structural layer recovers those edges by construction; the now-default AST is what makes that recovery complete.) **(2) Necessary ≠ sufficient.** We never measure whether a region *helps an agent fix* a bug (Phase 4: a patch passing the hidden tests) — only whether the relevant files are retrievable. **(3) Seed heuristic.** The harness injects the fault at the highest-out-degree changed file, not at the genuinely failing test, so it does not exercise the regime where a symptom is lexically far from its cause — the regime most favourable to causal selection. **(4) Scale and statistics.** Three single-crate repositories, $n \approx 20$ each; differences are reported with their standard deviation but not cross-validated. **(5) Engine.** The parser now *defaults* to a real syn AST — measured 36.5% more accurate than the former line heuristic on the engine’s own

source (two-thirds vs. full import recall), the heuristic kept only as a fallback for non-Rust input — and ingestion was hardened to reconstruct bit-identically across processes (sorted import resolution; order-invariant centrality). Regions remain file-granular and merge only on explicit cross-file edges; scoring weights are hand-tuned, though off-by-default eigenvector-centrality and node-lifecycle (Stable/Working/Orphan) refinements are now available. None of these affect the *proven* properties (partition, determinism, replay) or the measured locality — but the empirical case for a downstream agent benefit is, at this point, *unproven*.

Conclusion

We set out to show that organising an agent’s memory by *causal regions* retrieves long-horizon context better than chunk retrieval, gave the construction a precise and deterministic definition, and proved it reconstructs bit-for-bit on replay. We then tested the retrieval claim against the obvious baseline on real data — and it did not hold: across 70 real bug-fix commits a plain lexical TF-IDF retriever ties causal selection and beats it at a tight budget, and a crash-trace pivot loses to RAG-over-the-error-message, because on real code a fix’s files and its error messages share vocabulary. We report the negative result rather than bury it. What survives is not a better retriever but a different kind of object: a *deterministic, replayable, auditable* working memory in which an agent’s exact context state can be rewound and replayed under different parameters — time-travel debugging that a probabilistic retrieval stack cannot offer. Whether that auditability, or a structured-context advantage at *generation* time (Phase 4), yields a measurable downstream gain remains open. We release the engine, the honest validation harness and this paper so the distinction between what is proven, what is measured, and what is refuted can be checked rather than asserted.

99 P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020. arXiv:2005.11401. A. Asai et al. Self-RAG: Learning to Retrieve, Generate and Critique through Self-Reflection. *ICLR*, 2024. arXiv:2310.11511. D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. 2024. arXiv:2404.16130. F. Yamaguchi et al. Modeling and Discovering Vulnerabilities with Code Property Graphs. *IEEE S&P*, 2014. C. Packer et al. MemGPT: Towards LLMs as Operating Systems. 2023. arXiv:2310.08560. J. S. Park et al. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023. arXiv:2304.03442. LangChain. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. Software framework, 2023–2024. <https://github.com/langchain-ai/langgraph>. P. J. Denning. The Working Set Model for Program Behavior. *Communications of the ACM*, 11(5), 1968. N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 2024. arXiv:2307.03172. W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180. S. Haber and W. S. Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2), 1991. G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*, 2009. C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024. arXiv:2310.06770.

1. Causal Context Operating System (CCOS), an open research prototype. Source, reproduction scripts and this paper: <https://github.com/CHECKUPAUTO/CCOS>.↵